CalPolyPomona

MAT 4860
Intro to Math Modeling II
May 13th, 2026
Pomona, CA, USA

OPTIMAL DECISION-MAKING IN BLACKJACK USING STOCHASTIC MODELING AND DYNAMIC PROGRAMMING

Andy Le[1], Ethan Owusu-Bour[1], Anna Yee[1]

[1]California State Polytechnic University, Pomona

ABSTRACT

This project develops a stochastic model of a simplified blackjack game using probability theory and Markov chain techniques. The objective is to determine optimal player decision-making (hit versus stand) by modeling the game as a state-transition system driven by dynamic card probabilities. A transition matrix is constructed to represent the evolution of player totals under card draws, incorporating finite-deck effects and dealer uncertainty through weighted aggregation over possible hole cards. Multiple player strategies, including conservative, normal, risky, and probability-based ("smart") strategies, are evaluated through a computerized mathematical technique that models risks and uncertainty by repeatedly simulating a system using random sampling of input variables. Results demonstrate that probability-aware strategies outperform threshold-based approaches, showing how important it is to make use of dynamic, state-dependent decision-making. The model illustrates how stochastic processes and recursive expected value calculations can approximate optimal blackjack play.

1. INTRODUCTION

Blackjack is a widely studied stochastic game combining deterministic rules with probabilistic uncertainty [1]. Each decision (whether to hit or stand) depends on incomplete information about future card draws and the dealer's hidden card. From a mathematical perspective, blackjack can be modeled as a stochastic process in which states represent hand totals, transitions correspond to card draws, and probabilities evolve as cards are removed from a finite deck.
ht

This project applies Markov chain modeling to approximate optimal decision-making [2], [3]. Unlike classical infinite-deck assumptions, this model incorporates dynamic probabilities based on the current deck composition, making it more realistic but still computationally feasible for this level of research. Our goals are to:

- Construct a probabilistic model of blackjack
- Derive dealer outcome distributions
- Develop a transition matrix framework
- Evaluate decision strategies using expected value
- Compare strategies through simulation

Multiple simplifying assumptions were made to allow for the development of a workable model in the time given. The game will consist of one player and one dealer. The "shoe" will only contain one standard 52-card deck. The player will only have the option to hit or stand (no doubling or splitting). The player will bet a constant, arbitrary amount on each hand, so the results will track percentages of won hands per hands played, not an ending balance. The game will end once the shoe runs out.

2. MATHEMATICAL MODEL

2.1 State Space

We define the player state space as [3]:

$$S = \{2, 3, 4, \ldots, 21\}$$

Each state represents the player's current hand total. Note that the absorbing states are:

- 21: terminal winning state
- Bust: implicit absorbing state (total > 21)

2.2 Deck and Probabilities

Let the deck be represented as:

$$D = \{(c, n_c)\}$$

Where:

- c = card value (1–10),
- n = number of remaining cards of value c

Therefore, the total number of cards is:

$$N = \sum_c n_c$$

And the probability of drawing card c is [2], [5]:

$$P(c) = \frac{n_c}{N}$$

2.3 State Transitions

If the current state is j, drawing card c produces:

$$j' = \begin{pmatrix} j+11 & if\ c=1\ and\ j+11 \leq 21 \\ j+c & otherwise \end{pmatrix}$$

Thus the transition probability is:

$$P(j \to i) = \sum_{c:f(j,c)=i} P(c)$$

where $f(j, c)$ is the resulting total.

2.4 Transition Matrix

We define a transition matrix representation of the stochastic process [3]:

$$T \in \mathbb{R}^{20x20}$$

where:

- Columns = current state j
- Rows = next state i
- Entry:

$$T_{i,j} = P(state\ j \to i)$$

Properties:

- Column sums ≤ 1
- Missing probability corresponds to bust:

$$P_{bust}(j) = 1 - \sum_i T_{i,j}$$

2.5 Absorbing State

State 21 is absorbing in the Markov chain framework [3]:

$$T_{21,21} = 1$$

2.6 Dealer Model

The dealer follows deterministic rules:

- Hit on ≤ 16
- Stand on ≥ 17 (including soft 17)

However, uncertainty arises from the hole card. Let:

$$P(H = c) = \frac{n_c}{N}$$

Then the final transition matrix is:

$$T^{final} = \sum_c P(H = c) * T^{(c)}$$

where $T^{(c)}$ is the transition matrix after removing hole card c.

2.7 Expected Value Framework

For a player at state j, expected-value decision theory can be formulated as follows [4]:

- Stand value:

$$EV_{stand}(j) = P(win|j) - P(lose|j)$$

- Hit value:

$$EV_{hit}(j) = \sum_c P(c) * EV(F(j, c))$$

This defines a recursive structure for optimal decisions [4]:

$$EV(j) = max\{EV_{stand}(j), EV_{hit}(j)\}$$

3. METHODOLOGY/IMPLEMENTATION

3.1 Code to Math

Code Component	Mathematical Meaning
`deck`	Discrete probability distribution
`get_probabilities`	$P(c)$
`build_transition_matrix`	$T_{i,j}$
`build_final_transition_matrix`	Weighted mixture T^{final}
`play_player_turn`	Markov process evolution
`player_action`	Policy function

Listed above are some key components of the simulation program.

The **deck** is a dictionary that maps each card value to its remaining deck count. For example, if only one ace has been played so far, the (key: value) pair in the **deck** would be (1: 3), representing the card value and how many aces are left in the deck.

The function **get_probabilities** returns a dictionary that maps each card value to its probability of being drawn. Building on the previous example, calling **get_probabilities** on the current deck of 33 cards would return a dictionary containing the (key: value) pair (1: 0.091) as there is now a 9.1% percent chance the next card will be an ace.

The block **build_transition_matrix** iterates over the current state and every possible card draw to compute the new state and fill the transition matrix.

The function **build_final_transition_matrix** builds a transition matrix that accounts for every possible dealer starting position.

The function **play_player_turn** allows the Markov process to continue, updating hand totals and building a new final transition matrix at each hand.

Finally, **player_action** is the policy function that will map a current state to a player's action (hitting or standing). Note that the probabilities and transition matrices outlined previously are not used in this function unless the player is "smart". For the other three players, a simple threshold rule applies for this policy function.

3.2 Dynamic Probability Updates

After each card draw:

- Deck is updated
- Probabilities recomputed
- Transition matrix rebuilt

This creates a non-stationary stochastic process [2], since:

$$T = T(t)$$

depends on time (state of the deck).

3.3 Player Strategies

Four different player strategies are considered. The conservative player stands when their hand totals above 14, the normal player stands on any total above 16, and the risky player stands at any total above 17. It follows that if any of these three players are not standing, they are hitting. The smart player relies on the bust probability of the hand at the current deck state based on the transition matrix. Here, an arbitrary threshold is chosen, and the smart player will stand if $P_{bust} \geq 0.62$.

4. RESULTS

4.1 Simulation Results

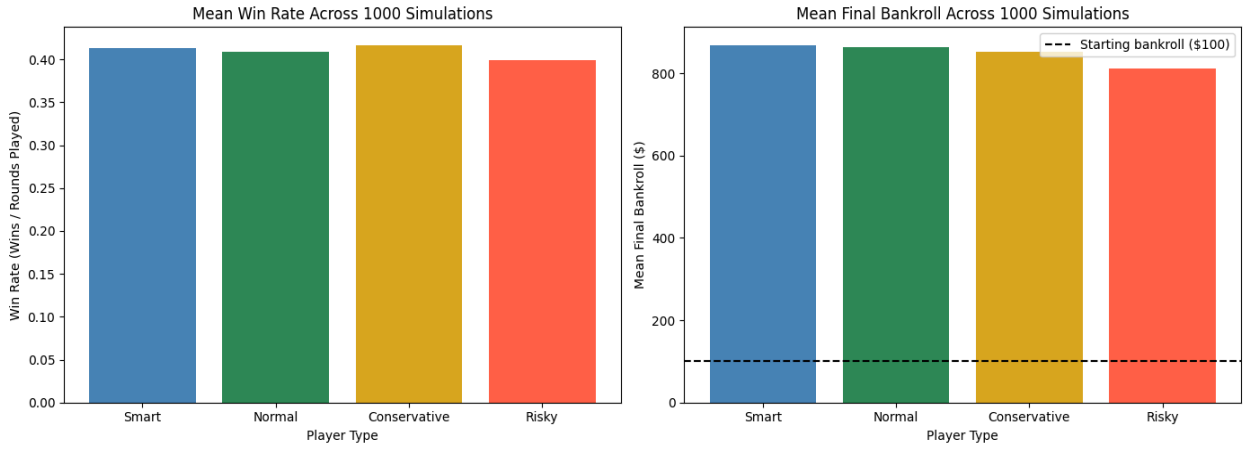

Figure 1. Mean win rate and final bankroll across 1000 simulations

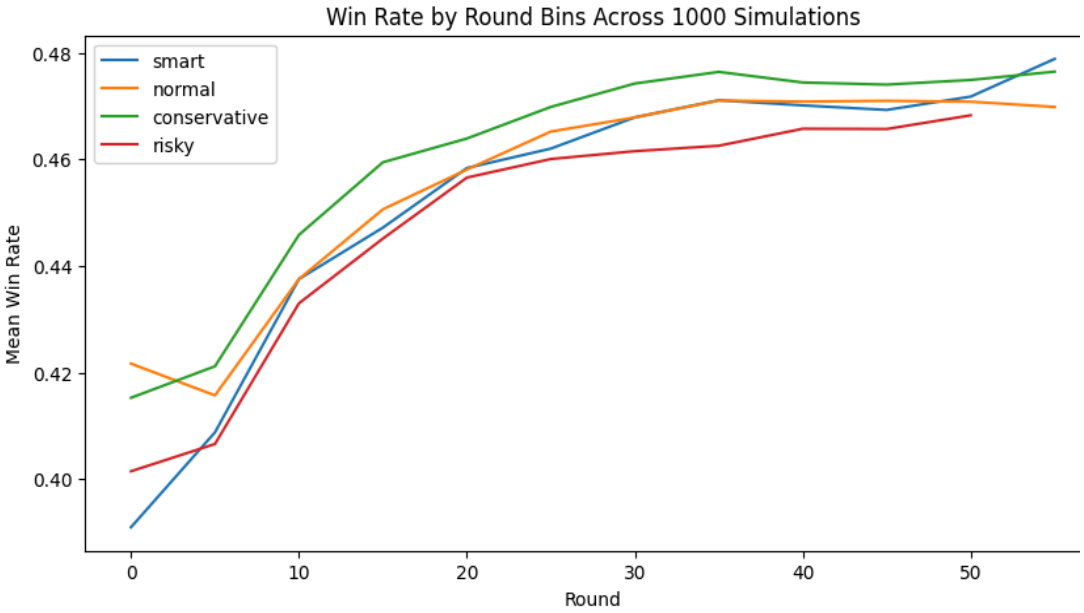

Figure 2. Simulation using 6x deck. Card counting is more effective as rounds increase.

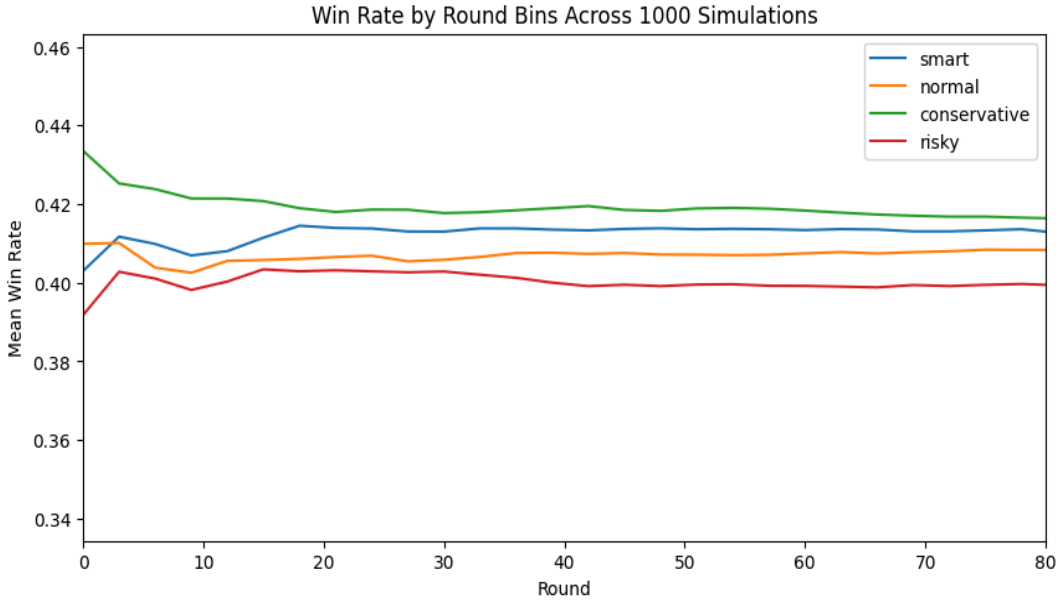

Figure 3. Simulation using 10x deck and increased bankroll. Card counting becomes less useful the more cards there are, so the performance of the smart player doesn't change much.

The plots show that the smart player achieves:

- Higher mean wins
- Higher win rate
- Stronger bankroll retention

4.2 Why the Smart Strategy Wins

The smart strategy's advantage comes from dynamic adaptation $P_{bust}(j)$ to the state of the deck. Unlike fixed strategies, the smart player responds in real time to the changing probability distributions in the remaining deck [1], [2]. In practice, this is the same information that a card counter keeps track of. Due to the smart strategy tracking the state of the remaining deck, the player moves between two behaviors. When the deck is rich in high cards, the bust probability increases, so the player will stand more. When the deck is rich in low cards, the bust probability lowers, so it is safer for the player to hit.

5. DISCUSSION

Blackjack can be modeled as a finite-state stochastic system. Transition matrices capture all possible future outcomes. Optimal decisions emerge from recursive expected value optimization methods commonly used in stochastic control theory [4]. This project highlights how a simple version of this method can meaningfully benefit overall player outcomes compared to fixed strategy.

It is shown that fixed strategies are suboptimal because they ignore deck composition and conditional probabilities. Even a simple probabilistic rule significantly improves performance. In this model, the smart strategy adapts a player's actions to the deck state. Ultimately, the smart player can understand when to play riskily and when to play conservatively, as opposed to the other players who are locked into a static action.

There are limitations to modeling a blackjack game this way. Many of the limitations come from the exact assumptions that made this model approachable to build. First, there is no full expected value computation because the model does not use full recursion. Secondly, this model simplifies the actions of the dealer and does not explicitly compute full outcome distributions. Next, there are no options to split or double, which are legal moves in a real game. Finally, a real casino will use a multi-deck shoe. This is important because a single deck experiences more dramatic shifts in deck probabilities, where a large shoe would be much less affected by the removal of cards.

6. FURTHER WORK

Our team has discussed the potential to implement this model into a multi-player game. In a busier casino, it is not guaranteed that a player will get to play heads up blackjack. This would include card competition and strategy interaction. A second natural extension point to this project would be incorporating dynamic bet sizing. To avoid detection, a card-counting player must play even when the deck is not favorable towards them. However, in order to risk less

money on these hands, the player can size down their bets until the deck becomes favorable. This would be implemented into the program right before the player's action is mapped and depend on the current heuristic value of the game state. Together, these extensions would allow the model to more closely reflect a real blackjack game.

7. CONCLUSION

This project demonstrates that blackjack can be effectively modeled using Markov chains and stochastic processes, with transition matrices capturing the probabilistic evolution of player states. By incorporating dynamic probabilities and recursive reasoning, even a simplified "smart" strategy significantly outperforms static approaches.

REFERENCES

[1] E. N. Thorp, *Beat the Dealer: A Winning Strategy for the Game of Twenty-One*, Revised Edition. New York, NY, USA: Vintage Books, 1966.

[2] S. M. Ross, *Introduction to Probability Models*, 12th ed. Amsterdam, Netherlands: Academic Press, 2019.

[3] J. R. Norris, *Markov Chains*. Cambridge, U.K.: Cambridge University Press, 1998.

[4] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, 4th ed., vol. 1. Belmont, MA, USA: Athena Scientific, 2017.

[5] C. M. Grinstead and J. L. Snell, *Introduction to Probability*, 2nd rev. ed. Providence, RI, USA: American Mathematical Society, 1997.